By combining all these attributes, you should find you're able to configure the model binder to bind to pretty much any request data your page handler wants to use. In general, though, you'll probably find you rarely need to use them; the defaults should work well for you in most cases.

That brings us to the end of this section on model binding. At the end of the model binding process, your page handler should have access to a populated binding model, and it's ready to execute its logic. But before you use that user input for anything, you must always *validate* your data, which is the focus of the second half of this chapter. Razor Pages automatically does validation for you out-of-the-box, but you have to actually check the results.

16.3 Validating binding models

In this section I discuss how validation works in Razor Pages. You already learned how important it is to validate user input in chapter 7, as well as how you can use `DataAnnotation` attributes to declaratively describe your validation requirements of a model. In this section you'll learn how to reuse this knowledge to validate your Razor Page binding models. The good news is that validation is built into the Razor Pages framework.

16.3.1 Validation in Razor Pages

In chapter 7 you learned that validation is an essential part of any web application. Nevertheless, minimal APIs don't have any direct support for validation in the framework; you have to layer it on top using filters and additional packages.

In Razor Pages, validation is built in. Validation occurs automatically after model binding but before the page handler executes, as you saw in figure 16.2. Figure 16.6 shows a more compact view of where model validation fits in this